

Lab03_hannah_nakamura

Hannah Miranda Nakamura (249491)

Laboratório 03

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.

```
library(readr)
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

`filter`, `lag`

Os seguintes objetos são mascarados por 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v forcats   1.0.1      v stringr   1.6.0
v ggplot2   4.0.3      v tibble    3.3.1
v lubridate 1.9.5      v tidyr     1.3.2
v purrr     1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag() masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
airlines <- read_csv('airlines.csv')
```

```
Rows: 14 Columns: 2
```

```
-- Column specification -----
Delimiter: ","
chr (2): IATA_CODE, AIRLINE
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

- Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;

```
airports <- read_csv(
  'airports.csv',
  col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
)
```

```
Rows: 322 Columns: 5
```

```
-- Column specification -----
Delimiter: ","
chr (3): IATA_CODE, CITY, STATE
dbl (2): LATITUDE, LONGITUDE
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

- Para o arquivo de vôos:
 - a. Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY;

```
colunas_flights <- cols_only(DESTINATION_AIRPORT = 'c', ARRIVAL_DELAY = 'd')
```

- b. Leia apenas 1 milhão de linhas por vez;
- c. Remova vôos em que o aeroporto de destino comece com a letra 1;
- d. Remova registros em que existam pelo menos uma coluna faltante;

- e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;

```
funcao_tratamento_flights <- function(input, pos) {  
  input |>  
  drop_na(DESTINATION_AIRPORT, ARRIVAL_DELAY) |> # (d)  
  filter(!str_starts(DESTINATION_AIRPORT, "1")) |> # (c)  
  group_by(DESTINATION_AIRPORT) |> # por aeroporto de destino  
  summarise( # (e)  
    soma_atraso = sum(ARRIVAL_DELAY), # soma de todos os atrasos  
    n_voos = n(), # número de vôos  
    .groups = "drop"  
  )  
}
```

```
flights <- read_csv_chunked(  
  'flights.csv.zip',  
  callback = DataFrameCallback$new(funcao_tratamento_flights), # (c, d, e)  
  chunk_size = 1e6, # (b)  
  col_types = colunas_flights # (a)  
)
```

- f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
media_atraso <- flights |>  
  group_by(DESTINATION_AIRPORT) |>  
  summarise(  
    media_atrasos = sum(soma_atraso)/ sum(n_voos),  
    .groups = "drop"  
  )  
  
media_atraso
```

```
# A tibble: 322 x 2  
  DESTINATION_AIRPORT media_atrasos  
    <chr>                <dbl>  
1 ABE                    5.80  
2 ABI                    4.08  
3 ABQ                    5.61  
4 ABR                   -3.39  
5 ABY                    8.68
```

```

6 ACK          4.91
7 ACT          10.6
8 ACV          9.68
9 ACY          11.8
10 ADK         -3.19
# i 312 more rows

```

2. Selecione a operação apropriada *join* para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

a. Identifique a coluna que é a chave na tabela `flights`;

Resposta: coluna `DESTINATION_AIRPORT`.

b. Identifique a coluna que é a chave na tabela `airports`;

Resposta: coluna `IATA_CODE`.

c. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?

```

flights_ausentes_airports <- flights |>
  anti_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

flights_ausentes_airports

```

```
# A tibble: 0 x 3
```

```
# i 3 variables: DESTINATION_AIRPORT <chr>, soma_atraso <dbl>, n_voos <int>
```

d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;

Resposta: comando `left_join`.

e. Armazene a tabela resultante no objeto `flights`.

```

flights <- flights |>
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
aerop_estados <- flights |>
  group_by(STATE) |>
  summarise(
    n_aeropostos = n()
  )

aerop_estados
```

```
# A tibble: 54 x 2
  STATE n_aeropostos
  <chr>      <int>
1 AK          102
2 AL           30
3 AR           24
4 AS            6
5 AZ           24
6 CA          121
7 CO           57
8 CT            6
9 DE            2
10 FL          102
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.
 - a. Carregue o pacote `leaflet`;

```
library(leaflet)
```

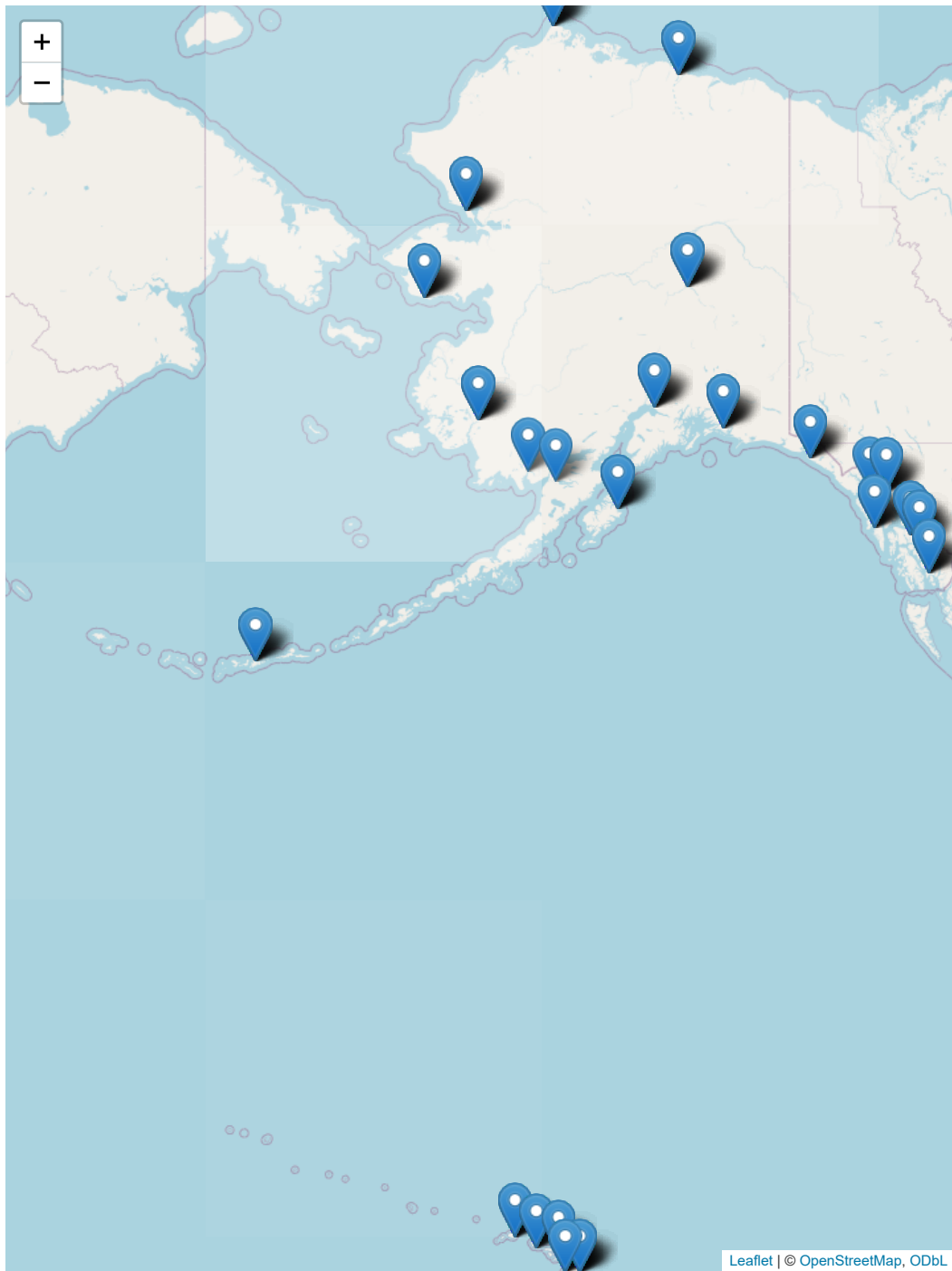
Warning: pacote 'leaflet' foi compilado no R versão 4.6.1

- b. Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
 - c. Armazene o grafico em b) numa variável chamada `this`;

```
flights <- flights |>
  drop_na(LATITUDE, LONGITUDE)

this <- leaflet(flights) |>
  addTiles() |>
  addMarkers(
```

```
lng = ~LONGITUDE,  
lat = ~LATITUDE,  
popup = ~paste0(  
  "<b>Aeroporto:</b>",  
  DESTINATION_AIRPORT,  
  "<br><b>Atraso Médio:</b>",  
  round(soma_atraso/ n_voos, 2),  
  "min"  
)  
)  
  
this
```



- d. Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 08:42:03 -03"
```